Characterising Quantum Randomness under Classical Rejection Sampling in Quantum-Backend-as-a-Service Architectures: A Readout-Mitigated Hardware Study with a Fair-Dice Case Study

Önder Öztürk

***Abstract*—Quantum-backend-as-a-service (QBaaS) platforms expose raw quantum measurements through ordinary web APIs, where they are almost always post-processed *classically* before use—most often by rejection (conditioning) onto a desired support. We study how device noise propagates through such classical post-processing, both theoretically and on real hardware, using a production-grade construction of mathematically fair dice as an exactly analysable case study (deployed in the backgammon variant "Tavla"). Each die is three qubits: $H^{\otimes 3}$ yields a uniform superposition over eight basis states, and rejecting the outcomes 0 and 7 gives an exactly uniform law on $\{1,\dots,6\}$; two dice use the six-qubit product circuit. We prove a tight bound showing that rejection *cannot remove* a raw hardware bias and amplifies it by at most $1/p(A) \le \frac{4}{3}$, and a statistical-power analysis fixes the minimum bias each experiment can detect. On two 156-qubit IBM Heron processors (five runs of 4000 shots each) the post-rejection total-variation distance from uniform is 0.015 ± 0.006 and 0.032 ± 0.006; matrix-free readout-error mitigation roughly halves the bias on the noisier device (to 0.014 ± 0.004) but not on the cleaner one, where calibration noise dominates. We expose the sampler behind a thin QBaaS contract with explicit provenance/fallback metadata, and analyse the synchronous-fallback versus asynchronous-provenance trade-off. Seeded simulator experiments ($N = 50{,}000$ rolls; marginal $\chi^2 = 10.1$ on 5 d.o.f., $p = 0.07$; joint $\chi^2 = 26.8$ on 35 d.o.f., $p = 0.84$) confirm the ideal statistics, and the entire study is reproducible from the released package.**

***Index Terms*—Quantum-backend-as-a-service (QBaaS), quantum-classical interface, quantum random number generation, rejection sampling, readout error mitigation, total variation distance, statistical power, fair dice, IBM Quantum, Qiskit.**

I. Introduction

QUANTUM-backend-as-a-service (QBaaS) platforms now let ordinary applications obtain measurement samples from explicitly constructed quantum circuits over a web API [1]. In almost every such use the raw samples are not consumed directly: they are post-processed *classically*—scaled, conditioned, or rejected—to land in the support and distribution the application needs. This classical post-processing sits on the critical boundary between a noisy quantum device and a deterministic client, and how device noise propagates across that boundary determines whether the service actually delivers what it advertises.

We make this concrete with the most basic non-trivial instance: producing a *mathematically fair* six-sided die. Three qubits in uniform superposition give eight equiprobable outcomes; rejecting two of them conditions onto an exactly uniform $\{1,\dots,6\}$ law. The construction is small enough to analyse exactly yet exhibits the full QBaaS pipeline—circuit, cloud dispatch, classical rejection, fallback—and we deploy it in a real mobile game, the backgammon variant "Tavla", where two independent dice determine every legal move and even small biases accumulate over many games. The fair die is therefore our running *case study*, not the contribution; the contribution is a quantitative account of how rejection-based post-processing transports quantum-hardware noise, and the engineering contract that keeps the result auditable.

A naïve mapping—reduce the raw integer modulo 6—introduces a well-known structural bias in which some faces are twice as likely as others. Rejection sampling instead discards the two forbidden outcomes, making the conditional law exactly uniform in the ideal limit. The interesting question on real hardware is not this ideal guarantee but its *robustness*: a biased device produces biased raw samples, and we show (Lemma 1) exactly how much of that bias rejection inherits and amplifies.

This paper makes four contributions. First, we prove a tight bound (Lemma 1) on how classical rejection sampling transports hardware bias: the conditioned total-variation distance is at most $1/p(A) \le \frac{4}{3}$ times the raw one, so rejection can never remove device bias and amplifies it only by a bounded factor (Section IV). Second, we report a repeated multi-backend hardware characterisation—five runs on each of two IBM processors, with matrix-free readout-error mitigation—giving run-to-run variance and how much residual

Ö. Öztürk is with the Information Technology Department, Rectorate, Kütahya Health Sciences University, Türkiye (ORCID: 0000-0001-6460-9497).

A reproducibility package—source code, seeded data, and manuscript sources—is openly available at https://github.com/onder-ozturk/quantum-dice and archived at doi:10.5281/zenodo.20539619 (MIT-licensed).

bias is removable, together with a statistical-power analysis that fixes the minimum detectable bias of every experiment (Sections IX-D–IX-E). Third, we define a thin QBaaS service contract that preserves an identical mathematical specification across simulator and real hardware while exposing explicit provenance and fallback metadata, with an explicit treatment of the synchronous-fallback versus asynchronous-provenance trade-off (Section VIII). Finally, the fair-dice case study itself provides an exactly analysable three-qubit-per-die construction with a proof of uniformity and a side-by-side comparison against the biased modulo mapping (Sections III–IV), validated by large-scale seeded Aer experiments (Section IX) and deployed in a mobile Tavla game.

Related work includes the well-known IBM "Truly Quantum Dice" demonstration [2], the quantum random-number-generation surveys of Herrero-Collantes and Garcia-Escartin [3] and Ma et al. [4], and the circuit construction of Orts et al. [5] for sampling integers inside an arbitrary interval—of which our concrete $d = 6$ rule is a special case. Quantum games have previously been realised on a photonic one-way quantum computer [6] and on IBM-Q [7]. Classical treatments of unbiased sampling from biased sources date back to von Neumann [8], are discussed in detail by Knuth [9], and are codified for random-bit-generator constructions by NIST [10]. A strictly stronger goal—cryptographically *certified* randomness from quantum sampling—was put on rigorous footing by Aaronson and Hung [11] and recently demonstrated experimentally on a 56-qubit trapped-ion processor [12]; we emphasise that our construction makes *no* certification claim and provides no advantage over a classical generator for ordinary play. Recent benchmarks of gate-based quantum random-number generation on IBM superconducting hardware likewise report usable but low-throughput, high-cost-per-bit randomness [13], consistent with our own observation that a cloud quantum round-trip is, for ordinary dice, strictly more expensive than a local generator. Relative to this body of work our focus is different: not a new randomness primitive but the *characterisation* of a ubiquitous QBaaS pattern—classical rejection/conditioning of noisy quantum samples—made quantitative by a transfer bound (Lemma 1), a statistical-power analysis, and a repeated multi-backend mitigated study, with the exactly analysable fair-dice construction as the case study and a mobile deployment as the engineering contract.

II. Mathematical Preliminaries

Definition 1 (Computational basis)**.** *For a single qubit the computational basis states are denoted* $|0\rangle, |1\rangle$*. An* n*-qubit system has basis states* $|x\rangle$ *for* $x \in \{0, 1, \ldots, 2^n - 1\}$*, where the integer* x *is the binary representation of the bit string.*

The Hadamard gate (see, e.g., [14])

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{1}$$

creates an equal superposition when applied to a computational-basis state:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}. \tag{2}$$

Applying H to every qubit of the all-zero state yields the uniform superposition

$$H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle. \tag{3}$$

Measurement of this state returns each integer x with probability exactly $1/2^n$ (Born rule).

III. Case Study: A Fair Die from Three Qubits

The next four sections develop the case study—a fair six-sided die—that instantiates the QBaaS pipeline and that the hardware study later analyses. A six-sided die requires six equally likely outcomes. Three qubits naturally produce eight. We therefore interpret the measurement outcome $X \in \{0, \ldots, 7\}$ and map it to a die face via rejection sampling rather than reduction modulo 6.

The circuit for one die is simply

$$|\psi_d\rangle = H^{\otimes 3}|000\rangle = \frac{1}{\sqrt{8}} \sum_{x=0}^{7} |x\rangle. \tag{4}$$

Consequently $P(X = x) = 1/8$ for each x.

IV. Rejection Sampling and Exact Fairness

Define the valid set $A = \{1, 2, 3, 4, 5, 6\}$. A shot is accepted only when the raw outcome lies in A; otherwise it is rejected and the next shot in the batch is examined.

Proposition 1 (Exact fairness of a single die)**.** *Let* X *be uniform on* $\{0, \ldots, 7\}$*. Define the random variable* $D = X \mid X \in A$*. Then* $P(D = k) = 1/6$ *for every* $k \in A$*.*

Proof. By definition of conditional probability and the uniformity of X,

$$P(D = k) = P(X = k \mid X \in A) = \frac{P(X = k)}{P(X \in A)} = \frac{1/8}{6/8} = \frac{1}{6}. \tag{5}$$

□

The fairness above is exact only for an *ideal* uniform X. On real hardware the raw distribution is perturbed, and it is important to understand how rejection sampling transports that perturbation. The following bound, which we use to interpret the hardware results of Section IX-E, shows that rejection can never *remove* a bias and amplifies it by at most a constant factor.

Lemma 1 (Bias transfer under rejection)**.** *Let* p *be any distribution on* $\{0, \ldots, 7\}$ *with total-variation distance* $\delta = \mathrm{TVD}(p, u)$ *from the uniform law* u*, and let* $q(k) = p(k)/p(A)$*,* $k \in A = \{1, \ldots, 6\}$*, be the rejection-conditioned distribution. Then*

$$\mathrm{TVD}\big(q, \mathrm{Unif}(A)\big) \;\le\; \frac{\delta}{p(A)} \;\le\; \frac{4}{3}\,\delta + O(\delta^2), \tag{6}$$

and the first bound is tight.

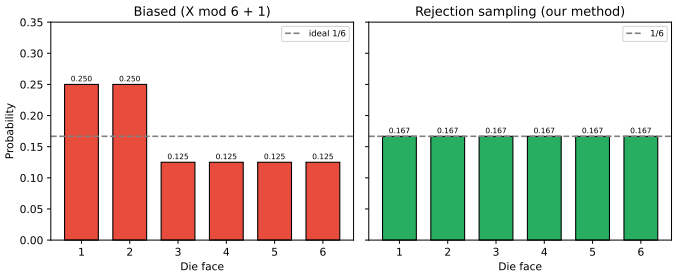

Fig. 1. Comparison of the biased modulo mapping with the exactly uniform distribution obtained by rejection sampling. Both start from a uniform random variable on $\{0, \ldots, 7\}$.

Proof. Write $a_x = p(x) - \frac{1}{8}$, so $\sum_x a_x = 0$ and $\sum_x |a_x| = 2\delta$. Put $P = p(A) = \frac{3}{4} + \varepsilon$ with $\varepsilon = \sum_{k\in A} a_k = -\sum_{x\notin A} a_x$. For $k \in A$,

$$q(k) - \tfrac{1}{6} = \frac{\frac{1}{8} + a_k}{P} - \frac{1}{6} = \frac{6a_k - \varepsilon}{6P}, \tag{7}$$

so

$$\mathrm{TVD}(q, \mathrm{Unif}(A)) = \frac{1}{12P}\sum_{k\in A}|6a_k - \varepsilon| \le \frac{\sum_{k\in A}|a_k| + |\varepsilon|}{2P}. \tag{8}$$

Since $|\varepsilon| \le \sum_{x\notin A}|a_x|$, the numerator is at most $\sum_x |a_x| = 2\delta$, giving TVD $\le \delta/P$. Also $|\varepsilon| \le 2\delta$ forces $P \ge \frac{3}{4} - 2\delta$, hence $\delta/P \le \delta/(\frac{3}{4} - 2\delta) = \frac{4}{3}\delta + O(\delta^2)$. Tightness: with $a_1 = a_2 = a_3 = t$, $a_4 = a_5 = a_6 = -t$ and $a_0 = a_7 = 0$ one has $P = \frac{3}{4}$, $\delta = 3t$, and $\mathrm{TVD}(q, \mathrm{Unif}(A)) = 4t = \delta/P$. □

The worst case is precisely a bias confined to the *accepted* faces (the rejected outcomes $\{0, 7\}$ unbiased); a bias confined to $\{0, 7\}$, by contrast, is removed by conditioning. Real devices lie in between: across our released runs the conditioned 1–6 TVD exceeds the raw 0–7 TVD by a factor of 1.2–1.3, comfortably below the worst-case $4/3$.

In contrast, the popular "modulo" mapping $D = (X \bmod 6) + 1$ produces a visibly biased distribution:

$$P(D=1) = P(D=2) = \frac{2}{8}, \qquad P(D=3) = \cdots = P(D=6) = \frac{1}{8}. \tag{9}$$

Figure 1 juxtaposes the two distributions.

The rejection rule is realised in a few lines of classical post-processing after measurement; see the `pick_valid_dice` routine in the accompanying implementation.

Table I situates the construction against the relevant prior art. We stress the honest conclusion it encodes: on fairness the method is *equivalent* to textbook classical rejection sampling and offers no statistical advantage over a cryptographically secure pseudo-random generator for ordinary play. Its only distinguishing property is that the underlying bits originate in a measured quantum state, giving auditable "quantum" provenance—at the cost of a network round-trip and, on real hardware, residual noise (Section IX-E).

V. Two Dice and Independence

Tavla (and standard backgammon) requires two independent dice. We encode both dice in a single six-qubit circuit:

$$|\Psi\rangle = H^{\otimes 6}|000000\rangle = \frac{1}{8}\sum_{x=0}^{7}\sum_{y=0}^{7}|x\rangle|y\rangle. \tag{10}$$

The first three qubits represent die 1 and the second three represent die 2. Because the state is a tensor product, the two registers are independent even before measurement. After applying the per-die rejection rule we obtain a pair (D_1, D_2) whose joint distribution is exactly uniform over the 6×6 grid. This independence is a property of the *ideal* product state; on hardware, crosstalk between physically adjacent qubits in the device coupling map can induce residual correlations between the two registers, so a deployment that relies on dice independence should verify it per backend (a systematic crosstalk study is left to future work). In practice this risk can be reduced at transpile time by mapping the two dice registers onto physically non-adjacent, well-separated qubits in the device coupling map, so that the dominant nearest-neighbour crosstalk is suppressed before it can correlate the registers.

Proposition 2 (Joint uniformity of two dice)**.** *Let $D_1 = X \mid X \in A$ and $D_2 = Y \mid Y \in A$. Then for every $i, j \in A$,*

$$P(D_1 = i, D_2 = j) = \frac{1}{36}. \tag{11}$$

Proof. The six-qubit state factors, therefore X and Y are independent and each uniform on $\{0, \ldots, 7\}$. The event that both lie in A has probability $(6/8)^2 = 9/16$. The conditional probability of any particular ordered pair is therefore

$$\frac{(1/64)}{(36/64)} = \frac{1}{36}. \tag{12}$$

□

VI. Generalisation and Batch Success Probability

For m dice the circuit uses $3m$ qubits and the per-shot acceptance probability is

$$p_m = \left(\frac{6}{8}\right)^m = \left(\frac{3}{4}\right)^m. \tag{13}$$

The expected number of shots until the first valid m-tuple is $\mathbb{E}[T_m] = 1/p_m = (4/3)^m$. For the two-dice case of interest, $\mathbb{E}[T_2] = 16/9 \approx 1.78$ raw shots (well below the batch size of 64 used in production). The probability that a complete batch of $B = 64$ shots contains *no* valid pair is

$$(1 - p_2)^{64} = \left(\frac{7}{16}\right)^{64} \approx 1.05 \times 10^{-23}, \tag{14}$$

which is negligible for all practical purposes.

The same recipe gives a fair d-sided die for any d: take $k = \lceil \log_2 d \rceil$ qubits, accept the first d of the 2^k equiprobable outcomes and reject the remaining $2^k - d$. Table II lists the standard tabletop and role-playing dice; powers of two ($d = 4, 8$) need no rejection, while $d = 6, 10, 12, 20$ carry only a modest overhead.

TABLE I
COMPARISON WITH PRIOR ART. "FAIR $d6$" IS EXACT UNIFORMITY IN THE IDEAL (NOISE-FREE) MODEL. THE PRESENT METHOD MATCHES CLASSICAL REJECTION SAMPLING ON FAIRNESS; THE QUANTUM VARIANTS PAY A NETWORK ROUND-TRIP AND INHERIT DEVICE NOISE. "OVERHEAD" IS RELATIVE TO A LOCAL CRYPTOGRAPHICALLY SECURE PRNG; "TYPICAL LATENCY" IS THE ORDER-OF-MAGNITUDE WALL-CLOCK COST PER ROLL.

Method	Fair $d6$?	Bit source	Overhead vs CSPRNG	Typical latency
$H^{\otimes 3}$ then $(\cdot \bmod 6)$	no (biased)	quantum	—	seconds (cloud)
$H^{\otimes 3}$ + rejection (*this work*)	yes (ideal)	quantum	network round-trip + device noise	seconds (cloud)
von Neumann/Knuth rejection [8], [9]	yes	classical	none beyond the entropy source	sub-μs (local)
Arbitrary-interval circuit [5]	yes (general d)	quantum	network round-trip + device noise	seconds (cloud)
CSPRNG + rejection [10]	yes	classical	cheapest (entropy source only)	sub-μs (local)

TABLE II
GENERALISATION TO A FAIR d-SIDED DIE. $k = \lceil \log_2 d \rceil$ QUBITS GIVE 2^k EQUIPROBABLE OUTCOMES; THE FIRST d ARE ACCEPTED AND THE OTHER $2^k - d$ REJECTED, SO THE PER-DIE ACCEPTANCE PROBABILITY IS $p = d/2^k$ AND THE EXPECTED NUMBER OF RAW SHOTS PER ACCEPTED VALUE IS $1/p = 2^k/d$.

d	qubits k	states 2^k	rejected	$\mathbb{E}[\text{shots}] = 2^k/d$
4	2	4	0	1.00
6	3	8	2	1.33
8	3	8	0	1.00
10	4	16	6	1.60
12	4	16	4	1.33
20	5	32	12	1.60

Algorithm 1 Quantum dice roll served by the QBaaS endpoint (identical for the Aer simulator and IBM hardware).

Require: number of dice m; batch size B (default 64)
$n \leftarrow 3m$
build an n-qubit circuit with n classical bits
apply H to every qubit, then measure every qubit
for up to 4 batches **do**
  run the circuit for B shots (`memory=True`)
  for all bitstrings in the result memory **do**
    decode m groups of 3 bits into integers in $\{0, \ldots, 7\}$
    if all m integers lie in $\{1, \ldots, 6\}$ **then**
      return the m-tuple as the dice
    end if
  end for
end for
return secure classical fallback (last resort)

VII. ALGORITHM

The end-to-end procedure executed by the service—identical for the Aer simulator and IBM hardware—is given in Algorithm 1.

The quantum portion is a single layer of $3m$ Hadamard gates followed by $3m$ single-qubit measurements (gate depth 1 in the ideal parallel model; the measurement adds one further layer). Classical post-processing per batch costs $O(Bm)$, and for $m = 2$ a single batch of 64 shots almost always suffices.

VIII. QBAAS SERVICE ARCHITECTURE AND CONTRACT

The construction is delivered as a quantum-backend-as-a-service: a thin HTTP layer that turns raw cloud measurements into the classical values the client consumes, with explicit provenance metadata. The quantum sampling layer is deliberately isolated from the game engine. A lightweight FastAPI micro-service exposes a single endpoint

`GET /roll?backend=aer|ibm&dice=2.`

The response always contains the classical values 'd1', 'd2', the actual backend that served the result, a job identifier, the number of shots consumed, and a Boolean 'fallback' flag. When the client requests the IBM backend but the job times out or the hardware is unavailable, the service transparently returns an Aer result with `fallback=true`; a final last-resort path uses a classical cryptographic RNG. Consequently a roll requested as "quantum" may in practice be served by the simulator or by a classical generator: the explicit `backend` and `fallback` fields make this provenance auditable, but a genuine quantum-hardware origin is best-effort, not guaranteed. An application that must guarantee hardware provenance should replace this synchronous, fallback-on-timeout endpoint with an *asynchronous* submit/poll interface that returns only genuine hardware results—accepting seconds-to-minutes of queue latency—and should never label a fallback roll as "quantum." The synchronous design here deliberately trades that guarantee for the low, bounded latency a real-time game needs. The Android game engine never sees quantum state vectors; it receives ordinary integers that are fed directly into the classical Tavla transition function.

Figure 2 depicts the data flow and the mathematical invariant that is preserved across both back-ends.

IX. EXPERIMENTAL VALIDATION

All experiments use the exact production circuit (`build_dice_circuit`) and the same accept test as the production `pick_valid_dice`; the run is seeded (`seed_simulator= 42`) and is therefore bit-for-bit reproducible from the accompanying scripts. We collected $N = 50{,}000$ valid two-dice rolls on the Qiskit Aer simulator [15], corresponding to 100,000 individual die faces.

A. Marginal uniformity

The observed face counts were

$$(16521, 16600, 16783, 16701, 16446, 16949). \qquad (15)$$

A χ^2 goodness-of-fit test (5 degrees of freedom, no estimated parameters) against the uniform expectation of 16,666.67 per face yields

$$\chi^2 = 10.13, \qquad p = 0.072, \qquad \text{TVD} = 0.0043. \qquad (16)$$

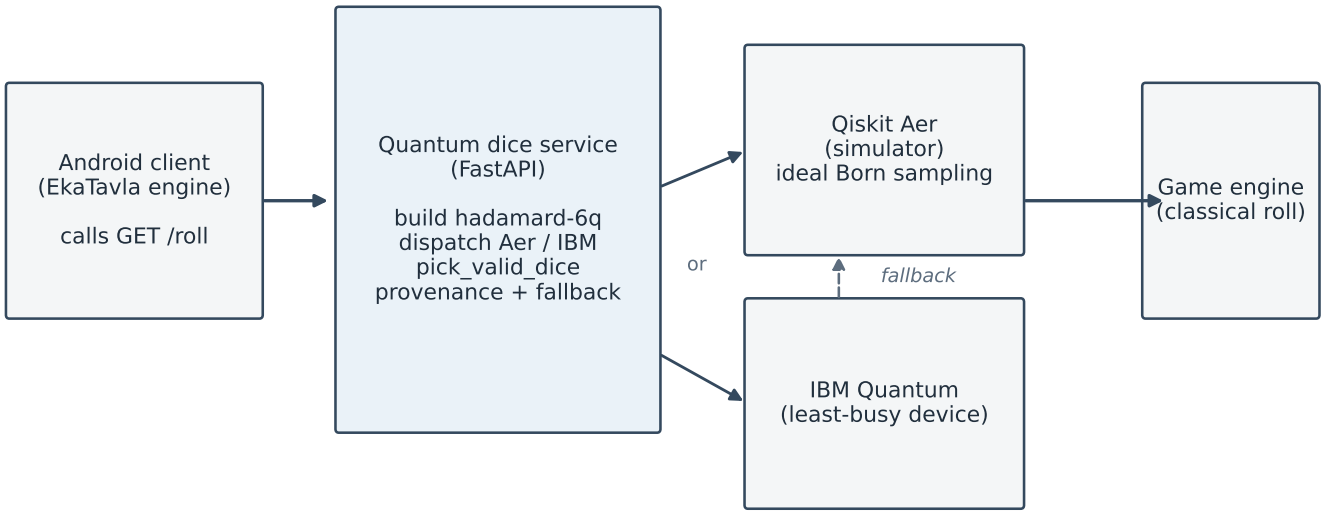

Fig. 2. Service architecture. The identical 'hadamard-6q' circuit and rejection-sampling post-processor are used for both simulator and real hardware. The game engine receives only classical die values together with provenance metadata.

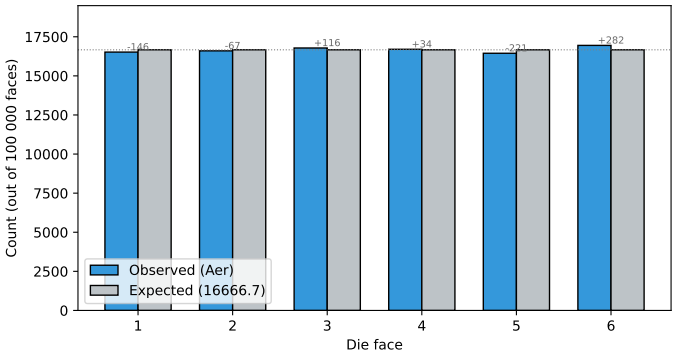

Fig. 3. Observed face frequencies from 50,000 Aer rolls (100,000 die faces). Residuals are at the level of statistical fluctuation; the χ^2 test does not reject uniformity ($\chi^2 = 10.1$, $p = 0.07$, TVD $= 0.004$).

The test does not reject uniformity at the 5% level; the largest single-face residual is $+282$ (about 2.4σ). Because the Aer simulator draws from precisely the Born distribution that Propositions 1–2 already guarantee, this experiment is best read as a sanity check on the (pseudo-random) sampling and bit-decoding pipeline, not as an empirical test of fairness on a physical device; a high p-value is failure to reject, not positive confirmation. Figure 3 shows the observed versus expected counts.

B. Joint uniformity

The 6×6 contingency table of (d_1, d_2) pairs was tested for joint uniformity over the 36 cells. The χ^2 statistic on 35 degrees of freedom is 26.85 with $p = 0.84$ and total-variation distance 0.0097 from the ideal $1/36$ per cell, again consistent with the predicted distribution. We stress that independence of the two dice holds *a priori* from the tensor-product structure of $|\Psi\rangle$ (Section V); this table is therefore a joint-uniformity goodness-of-fit check, not an empirical independence test, and on an ideal simulator it cannot reveal the qubit crosstalk that a real device might exhibit.

C. Acceptance rate

Direct sampling of 200,000 raw shots produced 112,687 valid pairs, giving an empirical per-shot acceptance probability of 0.56344, in close agreement with the theoretical value $(6/8)^2 = 0.5625$. The $50{,}000$ analysed rolls are the first $50{,}000$ accepted pairs of this same seeded run; returning the first accepted shot of a batch is unbiased because the raw shots are independent and identically distributed, so conditioning on "first accepted" preserves the uniform conditional law. With the production batch size $B = 64$ the expected number of raw shots per valid pair is $\mathbb{E}[T_2] = 16/9 \approx 1.78$, and the probability that a batch contains no valid pair is $(7/16)^{64} \approx 10^{-23}$ (Section V).

D. Statistical power and minimum detectable bias

Because a large p-value only signals *failure to reject*, we first quantify what bias each experiment can actually detect. For the χ^2 goodness-of-fit test on the six faces ($k = 6$ cells, 5 d.o.f., $\alpha = 0.05$) the critical value is $\chi^2_{0.95,5} = 11.07$, and attaining power $1 - \beta = 0.80$ requires a non-centrality $\lambda = 12.83$ (solving $\Pr[\chi^2_5(\lambda) > 11.07] = 0.80$). With N die faces the minimum detectable effect size is $w_{\min} = \sqrt{\lambda/N}$, where $w = \sqrt{\sum_k (p_k - \frac{1}{6})^2/(\frac{1}{6})}$. Since $w = \sqrt{6}\,\|p - u\|_2$ and $\mathrm{TVD} = \frac{1}{2}\|p - u\|_1$, Cauchy–Schwarz gives the two-sided conversion $w/(2\sqrt{6}) \le \mathrm{TVD} \le w/2$ (the upper bound attained by an equal $\pm$ split of the cells). The resulting minimum-detectable-TVD bands are: simulator ($N = 100{,}000$) $[0.0023, 0.0057]$; a single hardware run ($N \approx 4500$) $[0.011, 0.027]$; and five pooled runs ($N \approx 22{,}500$) $[0.0049, 0.012]$. This single calculation explains every p-value below: the simulator residual (TVD $= 0.0043$) and the cleaner

device's bias (Section IX-E, TVD ≈ 0.015) sit inside or below their detection bands and so are not rejected, while the noisier device's bias (TVD ≈ 0.032) sits above its band and is rejected in most runs.

E. Multi-backend hardware characterisation with readout-error mitigation

To probe how the ideal-circuit guarantee degrades on physical devices—and how much of that degradation is removable—we executed the identical `hadamard-6q` circuit on *two* independent IBM Quantum processors, `ibm_marrakesh` and `ibm_kingston` (both 156-qubit Heron devices), performing *five* independent runs of 4000 shots on each. On each backend we calibrated M3 matrix-free measurement-error mitigation [16] (the `mthree` package) once on the six measured qubits with 4000 shots, and recomputed every run's conditioned distribution from the mitigated quasiprobabilities. Reporting five runs lets us state run-to-run variance rather than a single, potentially unrepresentative number; per-job identifiers and full distributions are in the released data.

Table III and Fig. 4 summarise the result, and the two devices behave very differently. The raw rejection-conditioned 1–6 distribution on `ibm_kingston` is consistently skewed, TVD $= 0.032 \pm 0.006$ (mean $\pm$ s.d. over five runs), and the χ^2 test rejects uniformity in four of the five runs (with $p < 10^{-3}$ in all four). On `ibm_marrakesh` the raw bias is about half as large, TVD $= 0.015 \pm 0.006$, and uniformity is *not* rejected in any run (p from 0.06 to 0.88)—consistent with the power analysis above, since 0.015 lies inside the single-run detection band. The per-shot acceptance was 0.56 and 0.57 on the two devices, both close to the theoretical $9/16 = 0.5625$. The $\sim 40\%$ relative run-to-run spread on `ibm_marrakesh` is itself the point: a single run is not a reliable device characterisation.

Readout-error mitigation behaves as its purpose predicts, now consistently across runs. On the noisier device (`ibm_kingston`) M3 roughly halves the bias, from TVD $= 0.032 \pm 0.006$ to 0.014 ± 0.004. On the cleaner device (`ibm_marrakesh`), where the raw bias is already at the percent level, mitigation does *not* help—it marginally raises the mean ($0.015 \pm 0.006 \rightarrow 0.017 \pm 0.008$)—because the finite-shot sampling noise of the calibration is comparable to the small readout error it removes. After mitigation the two devices sit at a similar residual floor, TVD $\approx$ 0.014–0.017, indicating that the leftover bias is dominated by effects M3 does not address (state-preparation error and a residual, non-readout error floor) rather than by measurement error.

Two structural points follow, independent of the device. First, rejection sampling removes the *structural* modulo bias but cannot remove *hardware* bias: any skew present in the raw 0–7 samples is inherited by the conditional distribution—indeed Lemma 1 bounds the inherited bias by $\mathrm{TVD}_{\mathrm{raw}}/p(A) \leq \frac{4}{3}\mathrm{TVD}_{\mathrm{raw}}$, and readout mitigation only partially recovers it. Second, the residual unfairness on current hardware is at the percent level—small, but on `ibm_kingston` large enough to be statistically detected; a deployment that advertises "quantum dice" should therefore publish per-backend calibration and, where it matters, apply readout-error mitigation, rather than rely on the ideal guarantee.

TABLE III
REPEATED TWO-BACKEND HARDWARE CHARACTERISATION OF THE `HADAMARD-6Q` CIRCUIT: FIVE INDEPENDENT RUNS OF 4000 SHOTS EACH, BEFORE AND AFTER M3 READOUT-ERROR MITIGATION [16]. TVD IS THE TOTAL-VARIATION DISTANCE OF THE REJECTION-CONDITIONED 1–6 DISTRIBUTION FROM THE IDEAL $1/6$, AS MEAN $\pm$ S.D. OVER THE FIVE RUNS; "REJ." COUNTS RUNS WHOSE RAW χ^2 REJECTS UNIFORMITY AT $\alpha = 0.05$.

Backend	raw TVD	mit. TVD	raw rej.	accept.
`ibm_marrakesh`	0.015 ± 0.006	0.017 ± 0.008	$0/5$	0.56
`ibm_kingston`	0.032 ± 0.006	0.014 ± 0.004	$4/5$	0.57

The circuit diagram actually executed is shown in Fig. 5; the rejection flow is illustrated in Fig. 6.

X. DISCUSSION

The mathematical fairness guarantee holds only for the ideal circuit. On real IBM hardware, readout errors, decoherence, and gate infidelity will perturb the raw 0–7 distribution before rejection. Because rejection discards two out of eight outcomes, any bias already present in the hardware samples is inherited by the conditional distribution—quantitatively, Lemma 1 bounds the inherited bias by $\mathrm{TVD}_{\mathrm{raw}}/p(A) \leq \frac{4}{3}\mathrm{TVD}_{\mathrm{raw}}$ (measured in Section IX-E: raw TVD $= 0.015 \pm 0.006$ and 0.032 ± 0.006 on two backends over five runs each, only partially recoverable by readout-error mitigation). Consequently, production deployments that advertise "quantum dice" should either (a) remain on the simulator, (b) publish per-backend calibration data together with the observed TVD, or (c) apply additional classical conditioning/extraction. In our service the 'backend' and 'fallback' fields make the provenance of every roll explicit, allowing downstream auditing. Where mitigation is applied, its cost is modest in QBaaS terms: M3 needs a one-time per-backend calibration and only a small, matrix-free per-shot classical correction (sub-millisecond for six qubits), negligible against the seconds-scale cloud-queue latency—so readout-error mitigation is essentially free within the service's latency budget.

From a systems perspective the construction is deliberately over-engineered for a mobile game: a remote round-trip, possible queueing, and quota consumption are all incurred for randomness that a modern phone can generate locally at essentially zero cost and with cryptographic strength. The value of the present work therefore lies less in "better dice" than in (i) a clean pedagogical example of mapping a continuous quantum probability distribution onto a discrete uniform one, (ii) a concrete, reproducible demonstration of cloud quantum computing inside a consumer application, and (iii) an existence proof that the engineering surface between a quantum backend and a classical game engine can be reduced to a few dozen lines of well-specified code.

Future extensions could include device-independent certified randomness protocols, on-device quantum simulators for zero-latency play, or generalisation to other dice geometries

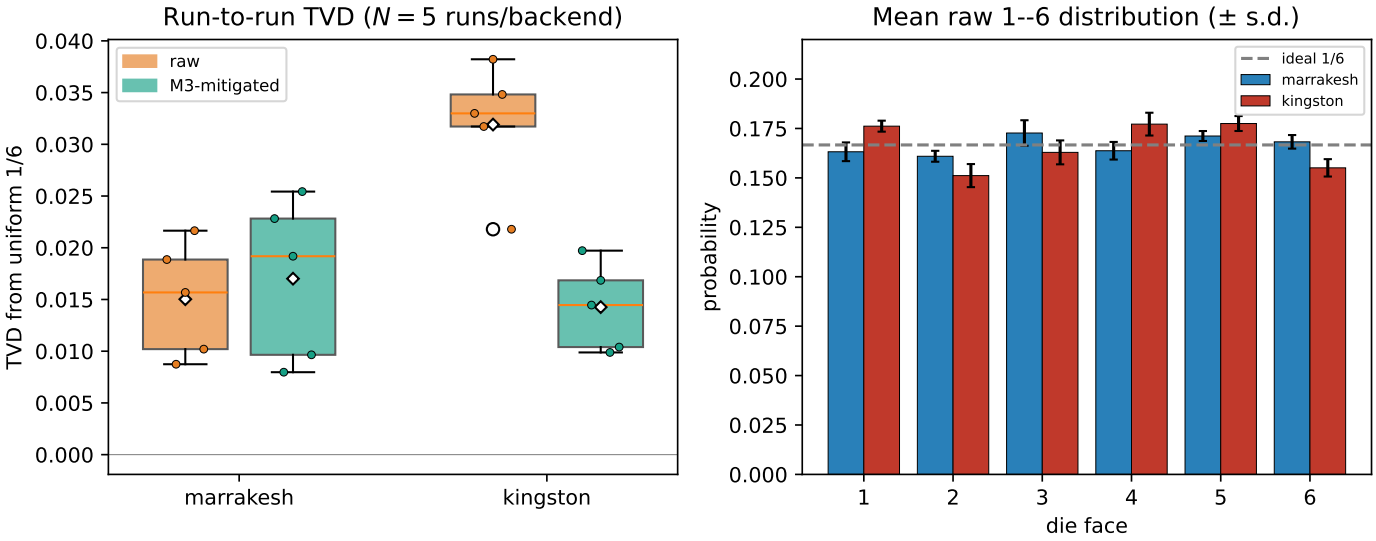

Fig. 4. Repeated hardware characterisation: five runs of 4000 shots on each 156-qubit Heron device. Left: run-to-run total-variation distance of the rejection-conditioned 1–6 distribution from uniform, raw versus M3-mitigated (boxes span the five runs; diamonds mark the mean; dots are individual runs). M3 roughly halves the bias on the noisier `ibm_kingston` but not on the cleaner `ibm_marrakesh`, where calibration sampling noise dominates. Right: the mean raw 1–6 distribution per device (± s.d.) against the ideal $1/6$.

(e.g., 4-, 8-, 10-, 12-, 20-sided) with optimal qubit counts and minimal rejection overhead.

Limitations. Three limitations bound the present claims. (i) The hardware evidence spans two backends with five runs each (Section IX-E); broader coverage—more devices, more runs, and the effect of qubit crosstalk on the product-state independence assumption—is left to future work. Following the protocol used here, we recommend that any follow-up report both raw and mitigated TVD across ≥ 5 runs per device, with bootstrap confidence intervals. (ii) The exactness result concerns the ideal circuit; under the fallback path a roll may be served by the simulator or a classical RNG, so end-to-end "quantum" provenance is best-effort. (iii) The contribution is a clean engineering-and-pedagogy construction rather than a new quantum primitive—for ordinary play a local CSPRNG is cheaper and cryptographically stronger, and the method provides no randomness *certification* in the device-independent sense.

XI. Conclusion

We have characterised how classical rejection sampling—a ubiquitous post-processing step in quantum-backend-as-a-service pipelines—transports quantum-hardware noise, using fair dice as an exactly analysable case study. A tight bound (Lemma 1) shows the conditioned bias is at most $1/p(A) \leq \frac{4}{3}$ times the raw bias, so rejection inherits but only mildly amplifies device noise. Although we derive it for a six-sided die, this transfer bound is not dice-specific: it governs *any* classical rejection or post-selection applied to noisy quantum samples—for instance variational post-selection or quantum-sampled cryptographic extraction—and so offers a reusable reference point for reasoning about the quantum–classical post-processing interface in QBaaS systems. Concretely, a single-layer Hadamard circuit on three qubits per die, followed by the rejection of two out of eight measurement outcomes, yields a mathematically exact uniform six-sided die in the ideal limit, and the construction extends without modification to any number of independent dice via a product state. Reproducible simulator experiments are consistent with the predicted statistics, with the caveat that an ideal simulator validates the sampling pipeline rather than the fairness guarantee itself. By exposing the sampler through a narrow, auditable HTTP contract we separate the quantum sampling concern from the classical game logic; the same contract also carries hardware-sourced bits, and five-run characterisations on two real IBM processors confirm that the guarantee degrades gracefully (percent-level bias, raw TVD 0.015–0.032) under device noise, with matrix-free readout-error mitigation recovering much of the bias on the noisier device; a tight transfer bound (Lemma 1) explains why rejection inherits but only mildly amplifies that bias. A larger multi-device characterisation remains future work.

The implementation, the seeded experimental data, the generation scripts, and the figure sources are included in the repository [17]; the seeded run makes every reported number reproducible bit-for-bit.

Reproducibility

All source code, the seeded experimental data, the analysis and figure scripts, and the manuscript sources are openly available [17] (tag `v1.8`, archived at doi:10.5281/zenodo.20539619, MIT-licensed). The simulator results are bit-for-bit reproducible: `generate_experiments.py` runs the exact production circuit with `seed_simulator=` 42. The hardware runs used Qiskit Runtime `SamplerV2` at the default resilience level, `optimization_level=` 1 with the standard `translator` translation stage, and 4000 shots per run; `run_ibm_experiment.py` (single-backend), `run_ibm_mitigated.py` (two-backend, mitigated), and `run_ibm_repeated.py` (the five-run, two-backend

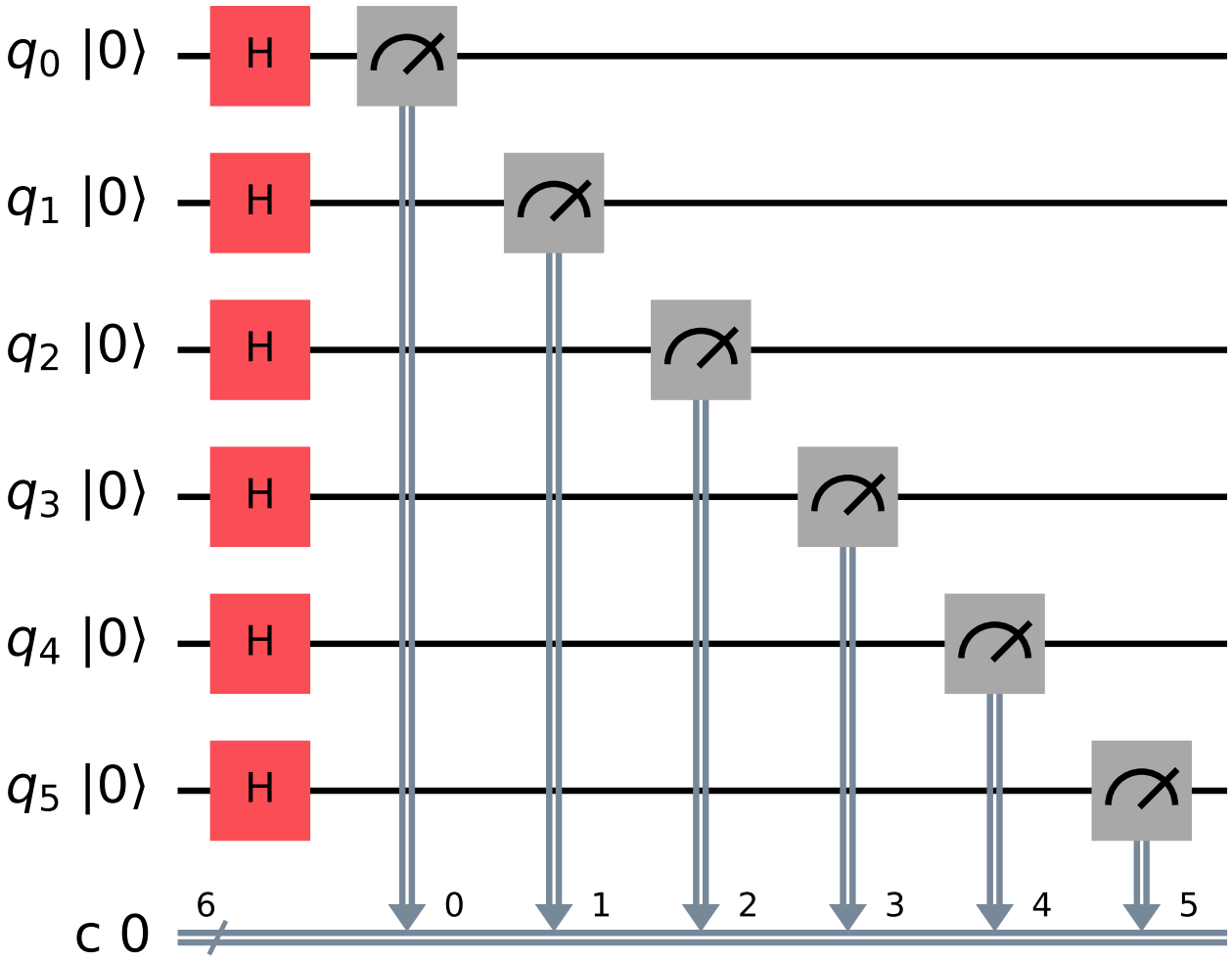

Fig. 5. The six-qubit circuit `hadamard-6q` used for every roll: six Hadamard gates followed by six computational-basis measurements (the classical register is bundled). Qubits q_0–q_2 encode die 1 and q_3–q_5 encode die 2; classical post-processing performs the per-die rejection test.

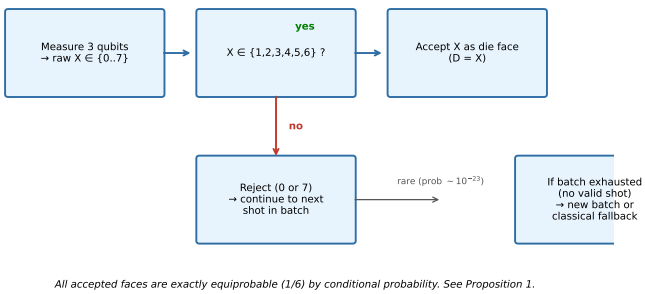

Fig. 6. Rejection sampling decision flow for a single shot. The only quantum operation is the uniform superposition and measurement; the accept/reject test is classical and extremely cheap.

characterisation reported here) cover the hardware path, with M3 mitigation provided by `mthree` calibrated on 4000 shots. Each script records the backend name, job ids, and the raw per-shot outcomes. Because hardware calibration drifts, exact device numbers are run-specific; the scripts and the saved JSON allow the procedure to be replayed on any backend.

Ethics Statement

This work is intended for entertainment and education. The construction is *not* suitable for applications that require certified or guaranteed randomness—such as gambling, lotteries, or cryptographic key generation: the fairness guarantee holds only for the ideal circuit, real-hardware output carries percent-level and device-dependent bias (Section IX-E), and the synchronous service may fall back to a classical generator, signalled by an explicit `fallback` flag (Section VIII). For those uses, a vetted, locally run cryptographic random-bit generator is the appropriate choice.

ACKNOWLEDGEMENTS

The author thanks the Qiskit open-source community; the Aer simulator made the reproducible experimental campaign straightforward, and the IBM Quantum Runtime primitives underpin the hardware path. The author used the `mthree` package for matrix-free measurement-error mitigation. The Android client and 3-D tabletop visualisation layers of Quantum Eka Tavla are outside the scope of this paper but provided the original motivation for a verifiable dice source.

REFERENCES

[1] IBM Quantum, "IBM quantum platform," https://quantum.ibm.com/, 2024, cloud access to superconducting quantum processors; least-busy scheduling and Qiskit Runtime.

[2] R. Huffman, "Truly quantum dice," https://medium.com/design-ibm/truly-quantum-dice-cfe372f4c586, 2018, iBM Design blog post demonstrating Hadamard-based quantum dice on IBM Quantum systems.

[3] M. Herrero-Collantes and J. C. Garcia-Escartin, "Quantum random number generation," *Reviews of Modern Physics*, vol. 89, no. 1, p. 015004, 2017.

[4] X. Ma, X. Yuan, Z. Cao, B. Qi, and Z. Zhang, "Quantum random number generation," *npj Quantum Information*, vol. 2, p. 16021, 2016.

[5] F. Orts, E. Filatovas, E. M. Garzón, and G. Ortega, "A quantum circuit to generate random numbers within a specific interval," *EPJ Quantum Technology*, vol. 10, no. 1, p. 17, 2023.

[6] R. Prevedel, M. Aspelmeyer, Č. Brukner, A. Zeilinger *et al.*, "Experimental realization of a quantum game on a one-way quantum computer," *New Journal of Physics*, vol. 9, no. 6, p. 205, 2007.

[7] F. Galas, "Quantum games on IBM-Q," in *Proceedings of the 2020 Federated Conference on Computer Science and Information Systems*, 2020.

[8] J. von Neumann, "Various techniques used in connection with random digits," *National Bureau of Standards Applied Mathematics Series*, vol. 12, pp. 36–38, 1951, classic paper introducing unbiased bit extraction from biased sources.

[9] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997, section on random number generation and common pitfalls with modulo bias when mapping to dice.

[10] NIST, "Recommendation for random bit generator (RBG) constructions," https://csrc.nist.gov/publications/detail/sp/800-90c/draft, 2016, nIST SP 800-90C (draft); discusses conditioning and extraction for random bit generators.

[11] S. Aaronson and S.-H. Hung, "Certified randomness from quantum supremacy," *arXiv preprint arXiv:2303.01625*, 2023, protocol for cryptographically certified randomness from sampling-based quantum advantage.

[12] M. Liu, R. Shaydulin, P. Niroula, M. DeCross *et al.*, "Certified randomness using a trapped-ion quantum processor," *Nature*, vol. 640, 2025.

[13] M. M. Louamri, A. Boussahi, N. E. Belaloui, and A. Tounsi, "A study of gate-based and boson sampling quantum random number generation on IBM and Xanadu quantum devices," *arXiv preprint arXiv:2507.03823*, 2025.

[14] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[15] IBM Quantum, "Qiskit: An open-source framework for quantum computing," https://www.ibm.com/quantum/qiskit, 2024, version 1.x; documentation and runtime primitives (SamplerV2).

[16] P. D. Nation, H. Kang, N. Sundaresan, and J. M. Gambetta, "Scalable mitigation of measurement errors on quantum computers," *PRX Quantum*, vol. 2, no. 4, p. 040326, 2021.

[17] Ö. Öztürk, "Quantum Tavla dice: Service, reproduction scripts, and data," https://github.com/onder-ozturk/quantum-dice, 2026, mIT-licensed reproducibility package: the hadamard-6q circuit and rejection sampling (dice.py), the Aer and IBM runners (backends.py, run_ibm_experiment.py), and the seeded, bit-for-bit reproducible experiment and figure scripts with saved data.